Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Bu derste Fatih Hoca'nın ders notlarındaki algoritma soruları çözülür. Her soruda önce basit bir çözüm kurulur, sonra işlem sayısı ve bellek açısından nasıl iyileştirileceği tartışılır. Dersin sonunda bir C programının derleme aşamaları komut satırında gösterilir ve `scanf`'in tampon (buffer) sorunu bir öğrencinin kodu üzerinden incelenir.
- İşlem sayısını saymak
  - 1'den N'e kadar toplamı döngüyle bulan algoritmada her temel işlem (atama, karşılaştırma, toplama, artırma) bir birim kabul edilir. Gerçekte bu işlemlerin süreleri farklıdır, ama analizde eşit sayılır.
  - `i = 1` ataması 1 kez, `i <= N` karşılaştırması N + 1 kez (son kontrol döngüden çıkmak içindir), toplama N kez, atama N kez, `i++` N kez yapılır. Toplam: $4N + 2$.
  - N = 10 için 42, N = 1000 için 4002 işlem yapılır. İşlem sayısı N ile doğrusal (lineer) büyür.
  - Gauss formülü $\frac{N(N+1)}{2}$ ise N ne olursa olsun 4 işlem gerektirir (bir toplama, bir çarpma, bir bölme, bir atama). Grafiği sabit bir doğrudur.
- Bir yöntemi başka probleme uyarlamak
  - Bilimde biri bir yöntem bulur, başkası onun açığını yakalar ya da yöntemi başka bir yere uyarlar.
  - Gauss toplamı ($\sum$) yerine çarpım ($\prod$) sorulursa $1 \cdot 2 \cdots N = N!$ elde edilir. Algoritma aynıdır: başlangıç değeri 1 olur, toplama yerine çarpma yapılır.
- Stirling formülü (yaklaşık çözüm)
  - $N! \approx \sqrt{2\pi N}\left(\frac{N}{e}\right)^N$.
  - Hoca işlem sayısını yaklaşık $N + 4$ olarak sayar: birkaç çarpma, karekök ve bölme, sonra $\left(\frac{N}{e}\right)$'nin N kez kendisiyle çarpılması. Bu, $4N+2$'den az ama yine N ile doğrusal büyür.
  - Bellek açısından fark yoktur (iki değişken).
  - Asıl fark doğruluktur: sonuç yaklaşıktır. Problem kesin sonuç (exact solution) istiyorsa bu algoritma işe yaramaz. Yaklaşık sonucun yettiği durumlarda kullanılabilir. Hoca örnek olarak faktöriyelin türevinin ancak yaklaşık olarak alınabileceğini söyler.
- Doğrusaldan daha yavaş büyüyen fonksiyonlar
  - Logaritmik fonksiyon ($\log N$) doğrusaldan yavaş büyür. Sabit fonksiyon hiç büyümez. Hedef, N gibi büyüyen bir algoritmayı $\log N$ veya sabit hâle getirmektir.
- Hızlı üs alma (kare alarak üs alma)
  - N^{13}'ü bulmak için döngüyle 13 çarpma yapılabilir. Ama `r = r * r` ile tek adımda kuvvet ikiye katlanır: $N \to N^2 \to N^4$.
  - Yöntem: üssün ikilik açılımı kullanılır ($13 = 1101_2$). `r = 1` ile başlanır. Bitler soldan sağa okunur. Bit 1 ise `r = r * r * N`, bit 0 ise yalnızca `r = r * r` yapılır.
  - 13 için: N, N^3, N^6, N^{13}. 13 çarpma yerine 4 adım yeterlidir.
  - Adım sayısı $\log_2 N$ kadardır. 2^{64} adımlık bir iş 64 adıma iner. Ayrıntılar özyinelemeli (recursive) algoritmalar konusunda anlatılacaktır.
  - Bit kontrolü için `if` de kullanılabilir. `if` kullanmadan da yapılabilir: `t = N * bit` gibi bir yardımcı değişkenle (bit 0 ise çarpan etkisizleştirilir). Bu tür kodlara dallanmasız hesaplama (branchless computing) denir.
- Pi sayısının seriyle hesaplanması
  - $\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \cdots = \sum_{n=1}^{\infty}(-1)^{n-1}\frac{4}{2n-1}$. Formülü öğrenciler birlikte buldu.

  - Sonsuz terim hesaplanamaz. N terimde kesilir. Bu bir kesme hatasıdır.
  - $(-1)^{n-1}$ her terimde üs hesabı gerektirir. Bunun yerine işaret başka yollarla değiştirilir.
- Pi algoritmasının iyileştirilmesi
  - 1. sürüm: `i mod 2 == 1` ise topla, değilse çıkar. `if`'in iki kolunda da aynı `1/(2i-1)` hesaplanıyor.
  - 2. sürüm: ortak hesap `if`'ten önce bir `T` değişkenine alınır. İki kolda aynı işlem varsa bunu bir üste taşımak her zaman daha iyidir. Hoca, yeni başlayanların sık yaptığı bu hatanın gömülü sistemlerde önemli olduğunu söyler.
  - 3. sürüm: `if` tamamen kaldırılır. Bir işaret değişkeni `s` tutulur, her turda `s = -s` (veya `s = s * -1`) yapılır: `toplam = toplam + s * 4.0 / (2*i - 1)`.
  - Ek iyileştirme: `toplam = 4`, `i = 2`, `s = -1` ile başlanırsa ilk terim döngüden çıkar.
  - Trade-off: `if`'li sürüm daha çok işlem yapar ama ek değişken kullanmaz. İşaret değişkenli sürüm daha az işlem yapar ama bir değişken fazla kullanır.
  - Algoritmada temel işlemler (toplama, çarpma, atama, karşılaştırma) tercih edilir. `if` ve döngüler temel işlem değildir. `if`'i kaldırmak süre kazandırır.
- e^x'in seriyle hesaplanması
  - $e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!}$.
  - 1. sürüm: her terim için pay (x^i) ve payda ($i!$) iki ayrı iç döngüyle sıfırdan hesaplanır. Toplam döngü sayısı yaklaşık N^2'dir.
  - 2. sürüm: iki iç döngü tek döngüde birleştirilir. Kod kısalır, ama hoca işlem sayısının değişmediğini söyler.
  - 3. sürüm (Tarık'ın önerisi): pay ve payda her turda sıfırlanmaz, bir önceki turdaki değer kullanılır: `pay = pay * x`, `payda = payda * i`, `sonuc = sonuc + pay / payda`. Çünkü $x^n = x \cdot x^{n-1}$ ve $n! = n \cdot (n-1)!$'dir. Tek döngü yeter ve işlem sayısı N^2'den N'e iner. Ek bellek gerekmez.
  - Çalışma soruları: $\cos x$ (radyan cinsinden), $\sqrt{x}$ ve $x^{y/2}$ için de benzer yaklaşık algoritmalar yazılabilir.
- Fibonacci sayıları
  - Dizi 1, 1, 2, 3, 5, 8, ... diye gider. Her terim kendinden önceki iki terimin toplamıdır. Bazı kaynaklarda dizi 0 ile başlar.
  - Dizi (array) ile çözüm: `F[1] = F[2] = 1`, `F[i] = F[i-1] + F[i-2]`. Döngü N kez döner, ama N elemanlı bir dizi gerekir. N = 10 milyon için 10 milyonluk dizi ayrılır. Bellek açısından verimsizdir.
  - Diziye gerek yoktur, çünkü yalnızca son iki terim lazımdır. Üç değişkenle: `c3 = c1 + c2; c1 = c2; c2 = c3;`.
  - Bir öğrencinin önerisiyle iki değişkenle: `c2 = c1 + c2; c1 = c2 - c1;`. İkinci satırda `c2 - c1` eski `c2`'yi verir.
- Armstrong sayıları (basamaklarının küpleri toplamı kendine eşit olan sayılar)
  - 1'den 999'a kadar her sayı için basamaklar ayrılır: `a = i / 100`, `b = (i % 100) / 10` (veya `(i - a*100) / 10`), `c = i % 10`.
  - C'de tam sayı bölmesi sonucu aşağı yuvarlar (floor). Akış diyagramında bu $\lfloor x \rfloor$ ile gösterilir. $\lceil x \rceil$ ise yukarı yuvarlamadır (ceil).
  - `a*a*a + b*b*b + c*c*c == i` ise sayı yazdırılır.
- Karşıt örnekle (counterexample) çürütme
  - Sonsuz bir arama uzayında bir önermenin her durumda doğru olduğu tek tek denenerek gösterilemez. Ama tek bir karşıt örnek bulmak önermenin yanlış olduğunu kanıtlar.
  - Bu tür aramalarda ne zaman duracağı bilinmediği için `for` değil, bir bayrak (flag) değişkeniyle `while` kullanılır.
  - Örnek 1: "Her pozitif doğal sayı n için $n^2 + n + 41$ asaldır." n = 1, 2, 3 için 43, 47, 53 asaldır. n = 40 için $40^2 + 40 + 41 = 40 \cdot 41 + 41 = 41^2 = 1681$ asal değildir. Önerme çürür.
  - Örnek 2: "$a^4 + b^4 + c^4 = d^4$ eşitliğini sağlayan pozitif tam sayılar yoktur." Burada dört değişken olduğu için iç içe dört `while` ve bir bayrak kullanılır. Bu önerme 1988'de bir karşıt örnekle çürütülmüştür.
  - Örnek 3 (Goldbach): "2'den büyük her çift sayı iki asal sayının toplamıdır." 4 = 2 + 2, 6 = 3 + 3, 8 = 5 + 3, 18 = 13 + 5 = 11 + 7. Algoritma: çift sayılar 4'ten başlayarak

ikişer artırılır. Her sayı için ondan küçük asallar bulunup bir diziye konur. Sonra her asal sayıdan çıkarılır ve kalanın da dizide olup olmadığı aranır. Bu önerme hâlâ ne kanıtlanabilmiş ne de çürütülebilmiştir.

- C programının derleme aşamaları (komut satırında `gcc` ile)
  - Komut satırında `gcc` yazınca işletim sistemi programı bulunduğu klasörde ve `PATH` ortam değişkenindeki (environment variable) yollarda arar. Dev-C++ ile gelen `gcc`'nin klasörü (`...\Dev-Cpp\MinGW64\bin\`) `PATH`'e eklenmelidir. Derste yolların arasına yanlışlıkla konmuş bir boşluk yüzünden `gcc` bulunamadı. Boşluk silinince çalıştı.
  - Windows'ta bir klasörde Shift'e basılı tutup sağ tıklayınca "Open command window here" ile o klasörde komut satırı açılır.
  - `gcc deneme.c` çıktı adı verilmezse varsayılan bir ad (`a.exe`) kullanır. `-o ad` ile çıktının adı belirlenir. `-Wall` uyarıları açar. `-save-temps` ara dosyaları saklar.
  - Ön işleme (preprocessing, `.c` → `.i`): yorumlar silinir, makrolar yerine konur (ör. `MYSUM` yerine `a + b`), `#include` edilen dosyanın içeriği eklenir. 9 satırlık program 600-700 satıra çıkar.
  - Derleme (compilation, `.i` → `.s`): kod assembly'ye çevrilir (`mov`, `add` gibi komutlar). `printf` çevrilmez, yalnızca bir `call printf` görülür, çünkü zaten derlenmiştir.
  - Çeviri (`.s` → `.o`): assembly komutları makine koduna eşleştirilir. Obje dosyası hâlâ küçüktür (yaklaşık 1 KB), çünkü `printf`'in kodu içinde yoktur.
  - Bağlama (linking, `.o` dosyaları → `.exe`): kendi obje dosyamız ve kütüphanelerin derlenmiş kodu birleştirilir. Çalıştırılabilir dosya bu yüzden çok daha büyüktür (yaklaşık 125 KB).
  - Çalıştırılabilir dosya çalıştırılınca proses olur ve işletim sistemi tarafından yönetilir.
  - IDE'ler bu aşamaları bizim yerimize yapar. Ama arka planda neler olduğunu bilmek gerekir.
- `scanf` ve giriş tamponu
  - Klavyeden girilen her şey, Enter tuşu (`\n`) dahil, giriş tamponunda (`stdin`) birikir. `scanf("%d", &n)` sayıyı alır ama `\n`'yi tamponda bırakır.
  - Ardından `scanf("%c", &c)` veya `scanf("%[^\n]", ad)` gelirse tampondaki `\n`'yi alır ve kullanıcıya hiç giriş yaptırmadan devam eder.
  - Çözüm 1: biçimin başına bir boşluk koymak: `scanf(" %c", &c)`, `scanf(" %[^\n]", ad)`. Boşluk, tampondaki boşluk ve satır sonu karakterlerini atlar.
  - Çözüm 2: `fflush(stdin)` ile tamponu boşaltmak. Hoca bunu "en temiz yol" olarak önerir (ders sonrası nota bakın).
  - `%s` boşlukta durur. "Emir Çağrı" girilirse yalnızca "Emir" alınır, "Çağrı" tamponda kalır ve sonraki `scanf` tarafından okunur. Satırın tamamı için `%[^\n]` kullanılır.
  - Bu sorunların akış diyagramıyla ilgisi yoktur. Tamamen C'nin giriş-çıkış akışlarıyla (stream) ilgilidir. Ayrıntılar ikinci sınıfta görülecektir.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir algoritmayı iyileştirmek için önce işlem ve bellek maliyeti hesaplanır, sonra gereksiz tekrarlar ayıklanır. "Korteksi biraz çalıştırınca" çok ciddi iyileştirmeler sağlanabilir.
- `if`'in iki kolunda da yapılan aynı işlem `if`'in üstüne alınmalıdır.
- Özellikle 1'den N'e giden ve iç içe olan `for` döngülerinden kaçınılmalıdır. İç içe iki döngü kabaca N^2 işlem demektir.
- Yaklaşık çözümler her zaman işe yaramaz. Problem kesin sonuç istiyorsa yaklaşık algoritma "çöptür".
- Trade-off'un ne olduğunu herkes bilmelidir. Sınavda sorulursa bir paragrafla açıklanabilmelidir.
- C'de döngüler 0'dan başlamak zorunda değildir. Problem neyi gerektiriyorsa oradan başlatılır. 0'dan başlayan dizilerdir.
- Akış diyagramı, sözde kod ve program kodu üçü de bilinmelidir, çünkü bir algoritmayla karşılaşınca hangisiyle yazılmış olacağı belli değildir.
- Bu derste çizilen akış diyagramlarını evde kodlayın. Pi algoritmasının `if`'li ve işaret değişkenli sürümlerini büyük bir N (ör. 10.000) için çalıştırıp sürelerini karşılaştırın. Goldbach algoritmasını kodlayın. Hoca bir sonraki derste rastgele öğrencilere kodlarını soracağını söyler.

- Kodlama yaparak, hata yapıp düzelterek öğrenilir. Yalnızca video izleyip hiç kod yazmayan öğrenci hata yapmaz, hata yapmayan da öğrenemez. Bu yabancı dilde cümle kurmayı öğrenmeye benzer.

Kısa Tekrar Notları

- Döngüyle 1..N toplamı: $4N + 2$ işlem. Gauss: 4 işlem.
- Stirling: $N! \approx \sqrt{2\pi N}\,(N/e)^N$. Yaklaşıktır.
- Hızlı üs: üssün bitleri soldan sağa. Bit 1: `r = r*r*N`. Bit 0: `r = r*r`. $\log_2 N$ adım.
- Pi: $4 - 4/3 + 4/5 - \cdots$. İşaret için `s = -s`.
- e^x: `pay *= x; payda *= i; sonuc += pay/payda;` tek döngü.
- Fibonacci dizisiz: `c2 = c1 + c2; c1 = c2 - c1;`.
- Basamaklar: `i/100`, `(i%100)/10`, `i%10`.
- Karşıt örnek aramada `while` + bayrak.
- `gcc -save-temps`: `.c` → `.i` (ön işleme) → `.s` (assembly) → `.o` (makine kodu) → `.exe` (bağlama).
- `scanf`'te önceki Enter tamponda kalır: `" %c"`, `" %[^\n]"`.

Detaylı Açıklamalar

Pi programı

Derste yazılan son sürüm:

```c
#include <stdio.h>
#include <math.h>

int main()
{
    int N;
    int s = -1;
    double tp = 4;
    int i = 0;

    printf("N giriniz");
    scanf("%d", &N);
    for (i = 2; i <= N; i++) {
        tp = tp + s * (4.0 / (2 * i - 1));
        s = -s;
        printf("Sonuc=%lf --> Hata =%lf\n", tp, fabs(M_PI - tp));
    }
    return 0;
}
```

- `4.0` yazılır, çünkü `4 / (2*i - 1)` iki tam sayının bölümü olur ve 0 verir.
- `M_PI`, `math.h` içindeki pi sabitidir. Mutlak değer için tam sayılarda `abs`, ondalıklı sayılarda `fabs` kullanılır.
- Hoca önce sonucu `4 * tp` olarak yazdırdığı için yanlış değerler gördü. Başlangıç değeri zaten 4 olduğu için bu çarpım kaldırıldı.
- Ekrana yazdırma, hesaplamadan çok daha uzun sürer. 10.000 terimlik hesap hemen biter, ama her satırı yazdırmak ekranın uzun süre akmasına yol açar.

Ders sonrasında aynı döngü yazdırma yapılmadan denendi:

N	Sonuç	Hata
10	3.041840	0.099753
100	3.131593	0.010000
1000	3.140593	0.001000
10000	3.141493	0.000100

Hata, terim sayısıyla ters orantılı olarak ($\approx 1/N$) azalıyor. Bu seri pi'ye çok yavaş yaklaşır.

Hızlı üs alma örneği

5^{13} için $13 = 1101_2$. Bitler soldan sağa okunur:

Bit	İşlem	r
1	`r = r*r*5`	5^1
1	`r = r*r*5`	5^3
0	`r = r*r`	5^6
1	`r = r*r*5`	5^{13}

Derste kullanılan diğer algoritmalar

Ders sonrasında derste çizilen akış diyagramları C ile yazılıp `gcc -Wall` ile derlendi:

- e^x tek döngülü sürüm, x = 1.5 ve 15 terim için `exp(1.5)` ile aynı sonucu verdi (4.481689).
- Fibonacci'nin iki değişkenli sürümü de üç değişkenli sürümü de 10. terim için 55 verdi.
- 1-999 arasındaki Armstrong sayıları: 1, 153, 370, 371, 407.
- $n^2 + n + 41$'in asal olmadığı ilk n değeri 40 çıktı (1681 = 41 × 41).

Ders sonrası notlar

- **Hızlı üs almada okuma yönü:** Hoca anlatırken "sağdan başlıyoruz" dedi, ama tahtadaki ok ve yapılan adımlar (N, N^3, N^6, N^{13}) bitlerin soldan sağa (en anlamlı bitten) okunduğunu gösteriyor. Bu yöntemde okuma yönü önemlidir. Aynı adımlar sağdan sola uygulanırsa (1011 sırasıyla) 5^{11} elde edilir. Sağdan sola okuyan farklı bir sürüm de vardır, ama orada `r` yerine tabanın karesi alınır ve bit 1 olduğunda sonuç tabanla çarpılır.
- **Stirling ve işlem sayısı:** Hoca $\left(\frac{N}{e}\right)^N$'nin N çarpma gerektirdiğini varsayarak $N + 4$ buldu. Bu kuvvet, aynı derste anlatılan hızlı üs alma yöntemiyle yaklaşık $\log_2 N$ çarpmayla hesaplanabilir. Bu durumda Stirling formülü toplamda $O(\log N)$ işlem gerektirir.
- **Faktöriyelin türevi:** Faktöriyel yalnızca tam sayılarda tanımlıdır, bu yüzden doğrudan türevi yoktur. Reel sayılara genişletilmiş hâli Gamma fonksiyonudur ($\Gamma(n+1) = n!$). Türev bu fonksiyon üzerinden alınır. Stirling formülü de Gamma fonksiyonunun büyük değerler için yaklaşık değerini verir.
- e^x **serisi:** Hoca serinin adından emin olmadığını söyledi. Bu, e^x'in Maclaurin serisidir (0 noktası etrafındaki Taylor serisi).
- **Birleştirilen döngüler:** İki ayrı `1..i` döngüsünü tek döngüde birleştirmenin yalnızca kodu kısalttığı, işlem sayısını değiştirmediği söylendi. Gövdedeki işlemler aynı kalır, ama döngü kontrolü (karşılaştırma ve artırma) iki yerine bir kez yapıldığı için toplam işlem sayısı da azalır.
- $a^4 + b^4 + c^4 = d^4$**:** Bu, Euler'in bir varsayımıdır. 1988'de Noam Elkies bir karşıt örnek buldu: $2682440^4 + 15365639^4 + 18796760^4 = 20615673^4$. Aynı yıl Roger Frye bilgisayar aramasıyla en küçük çözümü buldu: $95800^4 + 217519^4 + 414560^4 = 422481^4$. Hoca bu sayıların "çok büyük olmadığını" söyledi. Ama dört iç içe döngüyle bu büyüklüğe kadar kaba kuvvetle aramak pratik değildir. Frye'ın araması akıllı eleme yöntemleriyle yapılmıştır.
- **`fflush(stdin)`:** C standardında `fflush` yalnızca çıkış akışları için tanımlıdır. `fflush(stdin)`'in davranışı tanımsızdır (undefined behavior). Derste kullanılan Windows derleyicilerinde tamponu boşaltır, ama Linux'ta (glibc) hiçbir şey yapmayabilir. Taşınabilir çözümler: biçimin başına boşluk koymak (`scanf(" %[^\n]", ad)`) ya da satır sonuna kadar karakterleri okuyup atmak (`while ((c = getchar()) != '\n' && c != EOF);`). Ders sonrasında `scanf("%d", &n); scanf(" %[^\n]", ad);` sırası "3", "Emir Cagri" girdisiyle denendi ve adın tamamı doğru okundu.

- **Derleme aşamalarının adları:** Hocanın notunda `.s` → `.o` aşaması “translation” diye geçiyor. Bu aşamaya genellikle “assembly” (birleştirme) denir ve bunu yapan programa assembler denir. Adımlar sırasıyla: preprocessing, compilation, assembly, linking.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.